DOCUMENTAZIONE DI TESTING

Il processo di verifica e validazione rappresenta una fase imprescindibile nello sviluppo di sistemi software affidabili e rispondenti alle attese degli stakeholder. Attraverso attività di testing strutturate e metodiche è possibile individuare discrepanze tra il comportamento effettivo del prodotto e quanto specificato nei requisiti, riducendo il rischio di malfunzionamenti in produzione e migliorando la manutenibilità del codice nel lungo termine.

Sebbene l'esaustività dei test sia concettualmente irrealizzabile — a causa dell'esplosione combinatoria degli input e della natura discreta del software — una progettazione intelligente delle strategie di prova permette di massimizzare la probabilità di rilevare difetti critici. Criteri quali la partizione in classi di equivalenza, l'analisi dei valori limite e la copertura strutturale del codice guidano la selezione di casi di test significativi, bilanciando efficacia ed efficienza.

Di seguito viene descritto nel dettaglio l'insieme dei test svolti nel contesto di sviluppo di VetCare, sistema di gestione per cliniche veterinarie, suddivisi sulla base delle interazioni implementate. Per ciascun metodo sono stati identificati scenari tipici e anomali, definiti in base al comportamento atteso in condizioni operative normali e al limite delle specifiche. L'esecuzione automatizzata di tali test, supportata dal framework JUnit 5, ha permesso non solo di confermare il corretto funzionamento delle feature implementate, ma anche di guidare iterativamente le attività di debug e raffinamento del codice, in un'ottica di miglioramento continuo della qualità interna del prodotto.

I test sono stati progettati applicando la tecnica delle classi di equivalenza e dei valori limite. Per ogni parametro di input, sono state definite partizioni valide e non valide basate sulle specifiche funzionali e sui vincoli di dominio. La generazione dei casi di test segue il criterio di copertura minima delle classi di equivalenza, assicurando che ogni partizione sia esercitata almeno una volta.

OPERAZIONI DI TESTING

ITERAZIONE 1:

1.1 Test su modello VetCare

Metodo: **inserisciNuovaAnagrafica(String nome, String codiceFiscale, String contatto)**

Lo scopo del testing effettuato è verificare che il suddetto metodo gestisca correttamente l'inserimento di un nuovo proprietario, distinguendo tra codice fiscale inesistente e già presente, e gestendo valori nulli.

Metodo di individuazione delle classi di equivalenza:

1. **Per il parametro codiceFiscale:**
   - CE1 (Valida – Non presente): {cf ∉ sistema}
   - CE2 (Valida – Già presente): {cf ∈ sistema}
   - CE3 (Non valida – Null): {cf = null} *(gestito implicitamente nel caso generale dei parametri null)*
   - CE4(Non Valida- stringa vuota): {cf = “”} (gestita in maniera analoga al caso precedente)
2. **Per il parametro nome e ccontatto:**
   - CE5 (Valida – Stringa non nulla): {nome ≠ null, “”}
   - CE6 (Non valida – Null): {nome = null}

- CE7(Non valida- stringa vuota):{nome = “”}

Analogamente varrà per il parametro contatto. Ci aspettiamo che nel caso di parametri non validi il sistema non proceda con la creazione dell’istanza. Per il caso in cui cf è già presente non si crea una nuova istanza ma si restituisce quella già esistente.

Strategia di test: La strategia implementata copre:

- Una combinazione di input tutte valide (CE1, CE5).
- La combinazione con codiceFiscale già presente (CE2, CE5).
- Una combinazione con un parametro non valido (nome nullo: CE1, CE6, CE7).

Casi di test implementati:

- **TC1 –** Inserimento con CF nuovo (CE1, CE5): Verifica la creazione di un nuovo proprietario.
- **TC2 –** Inserimento con CF duplicato (CE2, CE5): Verifica il riuso dell'istanza esistente.
- **TC3 –** Inserimento con nome nullo (CE1, CE6, CE7): Verifica la gestione di input non valido.

Metodo: **inserisciNuovoAnimale(...)**
Si pone come obiettivo del test quello di validare la creazione di un nuovo animale associato a un proprietario, con particolare attenzione al controllo del microchip e alla gestione di parametri non validi.

Metodo di individuazione delle classi di equivalenza:

1. **Per il parametro microchip:**
   - CE1 (Valida – Intero > 0 e non presente): {microchip > 0} ∧ {microchip ∉ sistema}
   - CE2 (Valida – Intero > 0 e già presente): {microchip > 0} ∧ {microchip ∈ sistema}
   - CE3 (Non valida – Intero ≤ 0): {microchip ≤ 0}
2. **Per i parametri nome, specie, razza, dataNascita, proprietario:**
   - CE4 (Valida – Non null): {parametro ≠ null}
   - CE5 (Non valida – Null): {parametro = null}

Strategia di test:

La strategia combina la copertura delle classi di equivalenza con il test dei valori limite per il microchip (valori 0 e negativi). Viene testata:

- Una combinazione di input tutte valide (CE1, CE4 per tutti gli altri parametri).
- La combinazione con microchip già presente (CE2, CE4).
- Una combinazione con parametri non validi (microchip negativo - CE3 e nome nullo - CE5).

Casi di test implementati:

- **TC1 –** Inserimento con microchip nuovo e valido (CE1, CE4): Verifica la creazione di un nuovo animale.
- **TC2 –** Inserimento con microchip duplicato (CE2, CE4): Verifica il riuso dell'istanza esistente.

- **TC3 –** Inserimento con nome nullo e microchip negativo (CE3, CE5): Verifica la gestione di input multipli non validi.

Metodo: **confermaRegistrazione()**
Si verifica che la procedura di conferma di registrazione salvi correttamente proprietario e animale, rendendoli ricercabili, e che non salvi nulla in caso di dati non validi.

Metodo di individuazione delle classi di equivalenza:

1. CE1 (Valida – Stato pre-registrazione valido): {proprietarioCorrente ≠ null} ∧ {animaleCorrente ≠ null} ∧ {dati(animaleCorrente) sono validi}
2. CE2 (Non valida – Stato pre-registrazione non valido): {proprietarioCorrente = null ∨ animaleCorrente = null ∨ dati(animaleCorrente) non validi}

Strategia di test:
Copertura delle due classi di equivalenza derivate dallo stato.

Casi di test implementati:

- **TC1 –** Conferma con stato valido (CE1): Verifica il salvataggio permanente e la ricercabilità.
- **TC2 –** Conferma con stato non valido (CE2): Verifica che non venga salvato alcun dato inconsistente.

Metodo: **NuovaVisita(int microchip, String anamnesi, String esame, String diagnosi, int idVisita)**
Si testa la creazione di una nuova visita associata a un animale identificato tramite microchip, inclusa la gestione di microchip inesistenti o non validi.

Metodo di individuazione delle classi di equivalenza e valori limite:

1. **Per il parametro microchip:**
   - CE1 (Valida – Presente nel sistema): {microchip > 0} ∧ {microchip ∈ sistema}
   - CE2 (Non valida – Non presente nel sistema): {microchip > 0} ∧ {microchip ∉ sistema}
   - CE3 (Non valida – ≤ 0): {microchip ≤ 0} *(Valore limite)*

Strategia di test:

Copertura delle tre classi principali per microchip e verifica del valore limite (0/negativo).

Casi di test implementati:

- **TC1 –** Microchip valido e presente (CE1, CE4): Verifica la creazione della visita.
- **TC2 –** Microchip non valido (≤ 0) (CE3): Verifica il fallimento con valore limite.
- **TC3 –** Microchip valido ma non presente (CE2): Verifica il fallimento con identificatore inesistente.

Metodo: **confermaVisita()**

Lo scopo del testing effettuato è verificare che il metodo gestisca correttamente il salvataggio permanente di una visita nella cartella clinica dell'animale, transitando la visita dallo stato "corrente" allo stato "storico" all'interno della lista delle visite completate.

Metodo di individuazione delle classi di equivalenza:

1. **Per lo stato del sistema rispetto alla visita corrente:**
    - CE1 (Valida -- Visita corrente attiva): {animaleCorrente ≠ null} ∧ {visitaCorrente ≠ null} ∧ {visitaCorrente appartiene all'animaleCorrente}
    - CE2 (Non valida -- Nessuna visita corrente): {animaleCorrente ≠ null} ∧ {visitaCorrente = null}
    - CE3 (Non valida -- Animale corrente diverso da quello della visita): {animaleCorrente ≠ animaleVisita} ∧ {visitaCorrente ≠ null} *(caso implicito: la visita appartiene ad altro animale)*
2. **Per l'effetto della conferma sulla visita corrente:**
    - CE4 (Valida -- Post-conferma): {visitaCorrente = null}
    - CE5 (Valida -- Post-conferma): {visite[] contiene nuova visita} ∧ {visite.size() > 0}

Strategia di test:

La strategia copre:

- Una combinazione di input tutte valide con visita corrente attiva (CE1)
- Verifica dello stato post-conferma: azzeramento della visita corrente (CE4)
- Verifica dell'avvenuto salvataggio nella lista delle visite (CE5)
- Verifica del comportamento in assenza di visita corrente (CE2)
- Verifica del comportamento con animale corrente diverso da quello della visita (CE3)

Casi di test implementati:

- **TC1 -** Conferma visita con stato valido (CE1, CE4, CE5): Verifica che la visita venga rimossa dallo stato corrente e correttamente archiviata nella lista delle visite della cartella clinica.
- **TC2 -** Conferma visita senza visita in corso (CE2): Verifica che il metodo lanci IllegalStateException quando si tenta di confermare una visita in assenza di una visita corrente attiva.
- **TC3 -** Conferma visita su animale diverso (CE3): Verifica che il metodo lanci IllegalStateException quando l'animale corrente non corrisponde a quello a cui è associata la visita corrente (visita appartiene ad altro animale).

Metodo: **getInstance()**

Lo scopo del testing effettuato è verificare che il metodo implementi correttamente il pattern Singleton, garantendo che venga creata una sola istanza della classe VetCare durante l'intero ciclo di vita dell'applicazione.

Metodo di individuazione delle classi di equivalenza:

1. Per lo stato del sistema rispetto all'istanza:
    - CE1 (Valida -- Prima chiamata): {istanza = null}
    - CE2 (Valida -- Chiamate successive): {istanza ≠ null}

Strategia di test:

La strategia implementata copre entrambe le classi di equivalenza attraverso:

- Una prima invocazione del metodo per verificare la creazione dell'istanza (CE1)
- Una seconda invocazione per verificare che venga restituita la medesima istanza (CE2)

Casi di test implementati:

- TC1 -- Prima chiamata a getInstance() (CE1): Verifica che l'istanza restituita non sia null, confermando la corretta creazione dell'oggetto.
- TC2 -- Seconda chiamata a getInstance() (CE2): Verifica che l'istanza restituita sia la stessa della precedente attraverso assertSame, confermando l'unicità dell'istanza.

1.2 Test su modello CartellaClinica

Metodo: **inserisciVisita(String anamnesi, String esame, String diagnosi)**

L'obiettivo è verificare la corretta inizializzazione di una visita corrente e la gestione di input non validi che devono impedire la creazione della visita.

Individuazione delle classi di equivalenza:

1. ***Per i parametri (anamnesi, esame, diagnosi):***
   - CE1 (Valida): {Stringa non nulla e non vuota}
   - CE2 (Non valida - Null): {Stringa = null}
   - CE3 (Non valida - Vuota): {Stringa = ""}

Strategia di test: La strategia copre il caso ideale di inserimento e i casi limite per parametri nulli o stringhe prive di contenuto.

Casi di test implementati:

TC1 – Inserimento corretto (CE1): Verifica che con parametri validi la visita corrente non sia nulla.

TC2 – Inserimento nullo (CE2): Verifica che con parametri null la visita corrente non venga creata (risulti null).

TC3 – Inserimento stringa vuota (CE3): Verifica che con una stringa vuota ("") la visita non venga creata.

Metodo: **confermaVisita()**

Si convalida il passaggio della visita dallo stato "corrente" al salvataggio permanente nella lista delle visite della cartella clinica.

Individuazione delle classi di equivalenza:

- CE1 (Valida): {Visita corrente creata con dati validi}
- CE2 (Non valida): {Visita corrente non creata a causa di input nulli o vuoti}

Strategia di test: Verificare che il numero di visite salvate (dimensione della lista) aumenti solo in presenza di dati validi.

Casi di test implementati:

TC1 – Funzionamento corretto (CE1): Esecuzione di due inserimenti validi e verifica che il conteggio delle visite sia pari a 2.

TC2 – Gestione inserimenti invalidi (CE2): Tentativo di conferma di visite con parametri nulli o vuoti; verifica che la dimensione della lista non aumenti (mantenendo il valore degli inserimenti validi precedenti).

ITERAZIONE 2:

2.1 Test su modello Magazzino

Metodo: **getInstance()**
Lo scopo del testing effettuato è verificare che il pattern Singleton venga implementato correttamente, garantendo che esista una sola istanza della classe Magazzino nell'intero sistema.

Strategia di test: Verifica che l'istanza ottenuta non sia nulla e che richiami successive restituiscano la stessa identica istanza.

Casi di test implementati:

- TC1 – Verifica Singleton: Verifica che l'istanza non sia null e che due chiamate restituiscano lo stesso oggetto.

Metodo: **ricercaFarmaci(String nome)**
Lo scopo del testing effettuato è verificare che il metodo identifichi correttamente la presenza di un farmaco nel magazzino basandosi sul nome, con gestione case-insensitive e corretta gestione di valori non validi.

Metodo di individuazione delle classi di equivalenza:

1. Per il parametro nome:
    - **CE1 (Valida – Nome presente, case-insensitive):** {nome ∈ sistema} (indipendentemente dal case)
    - **CE2 (Valida – Nome assente):** {nome ∉ sistema}
    - **CE3 (Non valida – Null):** {nome = null}
    - **CE4 (Non valida – Stringa vuota):** {nome = ""}
    - **CE5 (Non valida – Stringa composta solo da spazi):** {nome = " "}

Strategia di test: La strategia implementata copre:

- Il caso di nome presente con diversi formati di case (CE1)
- Il caso di nome assente dal sistema (CE2)
- I casi limite di valori non validi (CE3, CE4, CE5)

Casi di test implementati:

- TC1 – Ricerca nome presente (CE1): Verifica che farmaci con nomi presenti vengano trovati indipendentemente dal case
- TC2 – Ricerca nome assente (CE2): Verifica che nomi non presenti restituiscano false
- TC3 – Ricerca con parametri non validi (CE3, CE4, CE5): Verifica che valori null, stringhe vuote o composte da soli spazi restituiscano false

Metodo: **selezionaFarmacoById(int id)**

Lo scopo del testing effettuato è verificare che il metodo recuperi correttamente un farmaco dal magazzino utilizzando il suo identificativo numerico, gestendo appropriatamente ID non validi, inesistenti e farmaci non più utilizzabili perché scaduti.

Metodo di individuazione delle classi di equivalenza:

1. **Per il parametro id:**
    - CE1 (Valida – ID presente e valido): $\{id > 0\} \wedge \{id \in sistema\} \wedge \{farmaco.scaduto = false\}$
    - CE2 (Valida – ID presente ma farmaco scaduto): $\{id > 0\} \wedge \{id \in sistema\} \wedge \{farmaco.scaduto = true\}$
    - CE3 (Valida – ID assente): $\{id > 0\} \wedge \{id \notin sistema\}$
    - CE4 (Non valida – ID ≤ 0): $\{id \leq 0\}$
2. **Per lo stato del farmaco (data di scadenza):**
    - CE5 (Valida – Farmaco non scaduto): $\{dataScadenza \geq dataCorrente\}$
    - CE6 (Non valida – Farmaco scaduto): $\{dataScadenza < dataCorrente\}$

Metodo di individuazione dei valori limite:

- **VL1:** ID = 1 (valore minimo positivo, tipicamente presente)
- **VL2:** ID = 0 (valore limite per transizione positivo/non positivo)
- **VL3:** ID = -1 (valore negativo)
- **VL4:** Data scadenza = dataCorrente (farmaco in scadenza oggi - ancora valido)
- **VL5:** Data scadenza = dataCorrente.minusDays(1) (farmaco scaduto ieri - primo giorno di non validità)

Strategia di test:

La strategia combina:

- Test di un ID valido e presente con farmaco non scaduto (CE1)
- Test di un ID valido e presente con farmaco scaduto (CE2, CE6)
- Test di un ID valido ma assente (CE3)
- Test dei valori limite per ID non validi (CE4, VL2, VL3)
- Test del comportamento atteso per farmaci scaduti

Casi di test implementati:

- **TC1 – ID valido e presente con farmaco non scaduto (CE1, CE5):** Verifica che venga restituito il farmaco corretto con attributi coerenti (id, nome, stato arrivo, mail fornitore) e che non venga lanciata alcuna eccezione.
- **TC2 – ID valido e presente con farmaco scaduto (CE2, CE6):** Verifica che il metodo lanci **FarmacoScadutoException** quando si tenta di selezionare un farmaco la cui data di scadenza è antecedente alla data corrente. Questo garantisce che farmaci non più utilizzabili non possano essere selezionati per operazioni cliniche.
- **TC3 – ID valido ma assente (CE3):** Verifica che venga lanciata **EntitaNonTrovataException** per ID inesistenti, segnalando l'assenza del farmaco richiesto.

- **TC4 – ID non valido (CE4):** Verifica che ID ≤ 0 lancino **EntitaNonTrovataException**, coprendo i valori limite (0, -5) e confermando che identificatori non positivi non siano mai considerati validi.

Metodo: **InserisciOrdine()**

Lo scopo del testing effettuato è verificare che il metodo gestisca correttamente l'inserimento di un nuovo ordine di farmaci, posizionando il farmaco nello stato "in attesa di arrivo" (arrivato = false) e rendendolo non ancora ricercabile tramite il metodo ricercaFarmaci() che opera solo sui farmaci effettivamente giunti in magazzino. Inoltre, si verifica la corretta gestione dei parametri di input non validi.

Metodo di individuazione delle classi di equivalenza:

1. **Per il parametro nomeFarmaco:**
    - CE1 (Valida -- Nuovo farmaco): {nomeFarmaco ∉ sistema} ∧ {nomeFarmaco ≠ null} ∧ {nomeFarmaco non vuoto}
    - CE2 (Valida -- Farmaco già esistente): {nomeFarmaco ∈ sistema}
    - CE3 (Non valida -- Null): {nomeFarmaco = null}
    - CE4 (Non valida -- Stringa vuota): {nomeFarmaco = ""}
    - CE5 (Non valida -- Stringa con soli spazi): {nomeFarmaco = " "}
2. **Per il parametro quantita:**
    - CE6 (Valida -- Intero positivo): {quantita > 0}
    - CE7 (Non valida -- Intero negativo): {quantita < 0} *(Valore limite: -1)*
    - CE8 (Non valida -- Intero zero): {quantita = 0} *(Valore limite)*
3. **Per lo stato del farmaco dopo l'inserimento:**
    - CE9 (Valida -- Stato atteso): {arrivato = false}
    - CE10 (Valida -- Mail fornitore): {mailFornitore = "fornitore@example.com"} *(valore predefinito)*
    - CE11 (Valida -- Quantità corretta): {quantitaFarmaco = quantitaInserita}

Strategia di test:

La strategia copre:

- Una combinazione di input tutte valide con farmaco non presente (CE1, CE6)
- Verifica dell'incremento della dimensione del magazzino
- Verifica che il farmaco NON sia visibile tramite ricercaFarmaci() in quanto non ancora arrivato (CE9)
- Verifica diretta nella struttura dati dei valori attesi (quantità, stato arrivo, mail fornitore) (CE10, CE11)
- Copertura completa delle classi di equivalenza non valide per entrambi i parametri:
    - Nome farmaco: null, stringa vuota, stringa con soli spazi (CE3, CE4, CE5)
    - Quantità: valori negativi e zero (CE7, CE8)

Casi di test implementati:

- **TC1 --** Inserimento ordine per nuovo farmaco con quantità valida (CE1, CE6, CE9, CE10, CE11): Verifica l'aggiunta del farmaco al magazzino, il corretto incremento della dimensione della mappa, l'impostazione dello stato "non arrivato", l'assegnazione della mail fornitore predefinita, la corretta quantità e la non visibilità tramite ricercaFarmaci().
- **TC2 --** Inserimento ordine con quantità negativa (CE1, CE7): Verifica che il metodo lanci IllegalArgumentException per quantità = -10.
- **TC3 --** Inserimento ordine con quantità zero (CE1, CE8): Verifica che il metodo lanci IllegalArgumentException per quantità = 0.
- **TC4 --** Inserimento ordine con nome farmaco null (CE3, CE6): Verifica che il metodo lanci IllegalArgumentException per nomeFarmaco = null.
- **TC5 --** Inserimento ordine con nome farmaco vuoto (CE4, CE6): Verifica che il metodo lanci IllegalArgumentException per nomeFarmaco = "".
- **TC6 --** Inserimento ordine con nome farmaco contenente soli spazi (CE5, CE6): Verifica che il metodo lanci IllegalArgumentException per nomeFarmaco = " ".

2.2 Test su modello Visita

Metodo: **creaTerapia(Farmaco farmaco, int posologia, String frequenza, LocalDate dataInizio, LocalDate dataFine)**
Lo scopo del testing effettuato è verificare che il metodo crei correttamente una terapia associata alla visita, con validazione completa di tutti i parametri e gestione di valori non validi.

Metodo di individuazione delle classi di equivalenza:

1. Per il parametro farmaco:
   - **CE1 (Valida – Non null):** {farmaco ≠ null}
   - **CE2 (Non valida – Null):** {farmaco=null}
2. Per il parametro posologia:
   - **CE3 (Valida – Intero positivo):** {posologia > 0}
   - **CE4 (Non valida – Intero ≤ 0):** {posologia ≤ 0}
3. Per il parametro frequenza:
   - **CE5 (Valida – Stringa non nulla e non vuota):** {frequenza≠ null} ∧ {frequenza.trim()≠""}
   - **CE6 (Non valida – Null):** {frequenza=null}
   - **CE7 (Non valida – Stringa vuota):** {frequenza=""}
   - **CE8 (Non valida – Stringa di soli spazi):** {frequenza.trim()=""} (Opzionale se gestito)
4. Per i parametri dataInizio e dataFine:
   - **CE9 (Valida – Entrambe non nulle e dataInizio ≤ dataFine):** {dataInizio≠ null} ∧ {dataFine≠ null} ∧ {dataInizio<dataFine}
   - **CE10 (Non valida – Data nulla):** {dataInizio=null ∨ dataFine=null}
   - **CE11 (Non valida – Intervallo incoerente):** {dataInizio > dataFine}

Metodo di individuazione dei valori limite:

- **VL1:** posologia=1 (valore minimo positivo)
- **VL2:** posologia=0 (valore limite tra positivo e non positivo)
- **VL3:** posologia=-1 (valore negativo)

Strategia di test: La strategia implementata copre:

- Il caso ideale con tutti i parametri validi (CE1, CE3, CE5, CE9)
- Tutte le singole condizioni di errore in isolazione
- Combinazioni di errori multiple
- Valori limite per la posologia

Casi di test implementati:

- **TC1 – Inserimento con tutti i parametri validi (CE1, CE3, CE5, CE9):** Verifica la corretta creazione della terapia con attributi coerenti
- **TC2 – Farmaco null (CE2, CE3, CE5, CE9):** Verifica che restituisca null quando il farmaco è null
- **TC3 – Posologia non valida (CE1, CE4, CE5, CE9):** Verifica che posologia ≤ 0 restituisca null (valori limite 0 e -5)
- **TC4 – Frequenza non valida (CE1, CE3, CE6/CE7, CE9):** Verifica che frequenza null o vuota restituisca null
- **TC5 – Date non valide (CE1, CE3, CE5, CE10/CE11):** Verifica che date nulle o intervallo incoerente (inizio > fine) restituisca null

Metodo: **stampaTerapia()**
Lo scopo del testing effettuato è verificare che il metodo generi correttamente una rappresentazione testuale della terapia associata alla visita.

Metodo di individuazione delle classi di equivalenza:

1. Per lo stato della terapia:
    - **CE1 (Valida – Terapia presente):** {terapia≠ null}
    - **CE2 (Valida – Terapia assente):** {terapia=null}

Strategia di test: La strategia verifica:

- Il messaggio appropriato quando non esiste terapia
- La generazione corretta della stringa quando la terapia è presente, inclusa la verifica di tutti i campi rilevanti

Casi di test implementati:

- **TC1 – Nessuna terapia (CE2):** Verifica che restituisca il messaggio "Nessuna terapia associata"
- **TC2 – Terapia presente (CE1):** Verifica che la stringa generata contenga tutti i dettagli della terapia (nome farmaco, posologia, frequenza)

2.3 Test su modello Laboratorio

Metodo: **getInstance()**
Lo scopo del testing effettuato è verificare che il pattern Singleton venga implementato correttamente per la classe Laboratorio, garantendo l'esistenza di una sola istanza in tutto il sistema.

Strategia di test: Verifica che l'istanza ottenuta non sia nulla e che chiamate successive al metodo restituiscano la stessa identica istanza.

Casi di test implementati:

- **TC1 – Verifica Singleton:** Verifica che l'istanza non sia null e che due chiamate restituiscano lo stesso oggetto.

Metodo: **produciEsame(String tipo, int microchip)**
Lo scopo del testing effettuato è verificare che il metodo crei correttamente un esame di laboratorio, accettando solo tipi validi e restituendo un identificativo univoco o un codice di errore.

Metodo di individuazione delle classi di equivalenza:

1. Per il parametro tipo:
    - **CE1 (Valida – "urine"):** {tipo = "urine"}
    - **CE2 (Valida – "sangue"):** {tipo = "sangue"}
    - **CE3 (Valida – "completo"):** {tipo = "completo"}
    - **CE4 (Non valida – Altro valore stringa):** {tipo ≠ "urine" ∧ tipo ≠ "sangue" ∧ tipo ≠ "completo"}
    - **CE5 (Non valida – Null):** {tipo = null}
    - **CE6 (Non valida – Stringa vuota):** {tipo = ""}
2. Per il parametro microchip:
    - **CE7 (Valida – Qualsiasi intero):** {microchip ∈ ℤ} (presumibilmente qualsiasi valore è accettato)
    - Nota: Non sono specificati vincoli per il microchip nel test

Metodo di individuazione dei valori limite:

- **VL1:** tipo = "urine" (valore valido, case-sensitive presumibilmente)
- **VL2:** tipo = "Urine" (valore con case differente - non testato, potrebbe essere non valido)
- **VL3:** tipo = "" (stringa vuota)
- **VL4:** tipo = " " (stringa di soli spazi - non testato)

Strategia di test: La strategia implementata copre:

- Tutti i tipi validi enumerati (CE1, CE2, CE3)
- Un tipo non valido generico (CE4)
- I casi limite di tipo null e stringa vuota (CE5, CE6)

Casi di test implementati:

- **TC1 – Tipi validi (CE1, CE2, CE3):** Verifica che "urine", "sangue" e "completo" restituiscano un ID > 0
- **TC2 – Tipo non valido generico (CE4):** Verifica che "xyz" restituisca -1
- **TC3 – Tipo null (CE5):** Verifica che null restituisca -1
- **TC4 – Tipo stringa vuota (CE6):** Verifica che "" restituisca -1

Metodo: **risultatiEsame(int microchip)**
Lo scopo del testing effettuato è verificare che il metodo recuperi correttamente tutti gli esami pendenti associati a un microchip, rimuovendoli dalla lista dei pendenti e spostandoli in quelli passati.

Metodo di individuazione delle classi di equivalenza:

1. Per lo stato del microchip:
    - **CE1 (Valida – Microchip con esami pendenti):** {microchip ∈ sistema} ∧ {esamiPendenti(microchip) > 0}
    - **CE2 (Valida – Microchip senza esami pendenti):** {microchip ∈ sistema} ∧ {esamiPendenti(microchip) = 0}
    - **CE3 (Valida – Microchip non presente):** {microchip ∉ sistema} (presumibilmente restituisce lista vuota)

Metodo di individuazione dei valori limite:

- **VL1:** Microchip con 1 esame pendente
- **VL2:** Microchip con 2+ esami pendenti
- **VL3:** Microchip con 0 esami pendenti
- **VL4:** Microchip mai inserito

Strategia di test: La strategia implementata copre:

- La creazione e recupero di esami pendenti (CE1)
- La verifica che dopo il recupero non rimangano esami pendenti (transizione CE1 → CE2)
- La verifica che esami recuperati vengano spostati nella lista degli esami passati

Casi di test implementati:

- **TC1 – Recupero esami pendenti (CE1):** Verifica che un microchip con esami pendenti restituisca una lista non vuota di dimensione corretta
- **TC2 – Verifica rimozione da pendenti (CE1 → CE2):** Verifica che dopo il recupero, lo stesso microchip non abbia più esami pendenti
- **TC3 – Verifica spostamento in passati:** Verifica che gli esami recuperati siano ora nella lista degli esami passati

2.4 Test su modello Calendario

Il testing della classe Calendario mira a garantire la corretta gestione degli appuntamenti della clinica, assicurando il rispetto dei vincoli temporali (orari lavorativi, durata minima) e l'assenza di conflitti (sovrapposizioni) tra le prenotazioni.

Metodologia Applicata: i test sono stati progettati per: Identificare scenari tipici e anomali basati sulle specifiche funzionali, garantire la copertura minima delle classi di equivalenza e verificare i comportamenti al limite delle specifiche (valori limite).

Metodo: **aggiungiAppuntamento(Animale a, String tipo, String desc, LocalDateTime inizio, LocalDateTime fine)**

Lo scopo del test è validare la creazione di un appuntamento, verificando che il sistema applichi correttamente i vincoli di business sugli orari e sulla durata.

Individuazione delle classi di equivalenza (Parametri temporali):

- **Per l'intervallo temporale:**

    - **CE1 (Valida):** {inizio < fine}
    - **CE2 (Non valida):** {fine < inizio}
- **Per la durata:**
    - **CE3 (Valida):** {durata ≥ 1 ora}
    - **CE4 (Non valida):** {durata < 1 ora}
- **Per la precisione dell'orario:**
    - **CE5 (Valida):** {minuti di inizio e fine = 00} (orario intero)
    - **CE6 (Non valida):** {minuti di inizio o fine ≠ 00}
- **Per l'orario lavorativo (8:00 - 18:00):**
    - **CE7 (Valida):** {orario compreso tra le 08:00 e le 18:00}
    - **CE8 (Non valida):** {inizio < 08:00 o fine > 18:00}
- **Per la coerenza della data:**
    - **CE9 (Valida):** {inizio e fine nello stesso giorno}
    - **CE10 (Non valida):** {inizio e fine in giorni diversi}

Strategia di test: La strategia copre il caso ideale di inserimento e tutte le violazioni dei vincoli che devono sollevare una IllegalArgumentException.

Casi di test implementati:

- **TC1 – Appuntamento Valido (CE1, CE3, CE5, CE7, CE9):** Verifica l'aggiunta di un appuntamento conforme a tutti i requisiti.
- **TC2 – Errori di Validazione (CE2, CE4, CE6, CE8, CE10):**
    - Verifica il fallimento se la fine precede l'inizio.
    - Verifica il fallimento per durate inferiori a 60 minuti.
    - Verifica il fallimento per orari non interi (es. 9:15).
    - Verifica il blocco di appuntamenti fuori orario lavorativo (prima delle 8 o dopo le 18).
    - Verifica il divieto di appuntamenti che scavalcano la mezzanotte.

Si verifica che il sistema impedisca la prenotazione di appuntamenti che confliggono temporalmente con quelli già esistenti.

Individuazione delle classi di equivalenza (Stato del Calendario):

- **CE1 (Valida – Nessun conflitto):** {L'intervallo [inizio, fine] è libero}
- **CE2 (Non valida – Sovrapposizione totale):** {L'intervallo coincide esattamente con uno esistente}
- **CE3 (Non valida – Sovrapposizione parziale):** {L'inizio o la fine cadono all'interno di un appuntamento esistente}
- **CE4 (Non valida – Inclusione):** {Il nuovo appuntamento contiene interamente quello esistente}

Valori Limite (Adiacenza):

- **VL1:** {Fine nuovo appuntamento = Inizio appuntamento esistente} (Consentito)
- **VL2:** {Inizio nuovo appuntamento = Fine appuntamento esistente} (Consentito)

Strategia di test: Assicurare che ogni tentativo di sovrapposizione sollevi una SovrapposizioneAppuntamentoException, mentre l'adiacenza perfetta sia permessa.

Casi di test implementati:

- **TC1 – Sovrapposizione Totale/Parziale (CE2, CE3, CE4):** Verifica che appuntamenti identici, che iniziano durante, finiscono durante o che "abbracciano" un impegno esistente vengano rifiutati.
- **TC2 – Verifica Adiacenza (VL1, VL2):** Verifica che sia possibile fissare un appuntamento che inizi esattamente nel momento in cui ne finisce un altro (adiacenza), garantendo la massima efficienza del calendario.

Metodo: **AggiungiTurno(MembroEquipe membro, LocalDateTime inizio, LocalDateTime fine)**

Lo scopo del testing effettuato è verificare che il metodo gestisca correttamente l'assegnazione di turni di lavoro ai membri dell'équipe, rispettando i vincoli temporali (orario di lavoro, durata minima, granularità oraria) e impedendo sovrapposizioni per lo stesso membro, pur consentendo turni contemporanei tra membri diversi.

Metodo di individuazione delle classi di equivalenza e valori limite:

1. **Per il parametro inizio e fine (granularità):**
   - CE1 (Valida -- Minuti a zero): {inizio.minuto = 0} ∧ {fine.minuto = 0}
   - CE2 (Non valida -- Minuti non zero): {inizio.minuto ≠ 0 ∨ fine.minuto ≠ 0} *(Valore limite: 15)*
2. **Per la durata del turno:**
   - CE3 (Valida -- Durata ≥ 1 ora): {(fine - inizio) ≥ 1 ora}
   - CE4 (Non valida -- Durata < 1 ora): {(fine - inizio) < 1 ora} *(Valore limite: 30 minuti)*
3. **Per l'orario di inizio (fascia lavorativa):**
   - CE5 (Valida -- Orario compreso tra 8:00 e 20:00): {inizio.hour ≥ 8} ∧ {fine.hour ≤ 20}
   - CE6 (Non valida -- Antecedente alle 8:00): {inizio.hour < 8} *(Valore limite: 7:00)*
   - *(Da testare: orario successivo alle 20:00 come ulteriore valore limite)*
4. **Per le sovrapposizioni:**
   - CE7 (Valida -- Nessuna sovrapposizione per il membro): {∄ turno ∈ turni[membro] con overlap(inizio, fine)}
   - CE8 (Non valida -- Sovrapposizione con turno esistente dello stesso membro): {∃ turno ∈ turni[membro] con overlap(inizio, fine)} *(Valore limite: inizio durante turno esistente)*
   - CE9 (Valida -- Sovrapposizione con turni di altri membri): {∃ turno ∈ turni[altriMembri] con overlap(inizio, fine)} *(comportamento ammesso)*
5. **Per il parametro membro:**
   - CE10 (Valida -- Membro non null): {membro ≠ null}

    - CE11 (Non valida -- Membro null): {membro = null} *(da testare)*

Strategia di test:

La strategia copre:

- Una combinazione di input tutte valide con membro diverso, orario valido, durata ≥ 1 ora e minuti a zero (CE1, CE3, CE5, CE7, CE10)
- Verifica del corretto funzionamento con membro diverso nello stesso orario (CE9)
- Copertura dei valori limite per:
    - Minuti non zero (CE2)
    - Durata inferiore all'ora (CE4)
    - Orario antecedente alle 8:00 (CE6)
    - Sovrapposizione parziale con turno esistente (CE8)

Casi di test implementati:

- **TC1 --** Aggiunta turno con parametri validi (CE1, CE3, CE5, CE7, CE10): Verifica che il turno venga correttamente aggiunto senza lanciare eccezioni.
- **TC2 --** Aggiunta turno con minuti non zero (CE2, CE3, CE5, CE7): Verifica che il metodo lanci IllegalArgumentException quando i minuti di inizio o fine non sono pari a zero (test con +15 minuti).
- **TC3 --** Aggiunta turno con durata inferiore a 1 ora (CE1, CE4, CE5, CE7): Verifica che il metodo lanci IllegalArgumentException quando la durata del turno è di soli 30 minuti (valore limite).
- **TC4 --** Aggiunta turno con orario antecedente alle 8:00 (CE1, CE3, CE6, CE7): Verifica che il metodo lanci IllegalArgumentException quando l'orario di inizio è alle 7:00 (valore limite).
- **TC5 --** Aggiunta turno in sovrapposizione con turno esistente dello stesso membro (CE1, CE3, CE5, CE8): Verifica che il metodo lanci IllegalArgumentException quando si tenta di assegnare un turno che si sovrappone (inizio durante turno esistente) allo stesso membro.
- **TC6 --** Aggiunta turno per membro diverso nello stesso orario (CE1, CE3, CE5, CE9, CE10): Verifica che sia consentito assegnare turni contemporanei a membri diversi dell'équipe.